

MNIST

◎ 구분 실습수업

MNIST 데이터 다운로드 받기

MLP_model.py 파일을 하나 생성하자.

그리고 여기에 데이터 다운로드 받고 나중에 모델도 정의해 줄 것이다.

```
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# 데이터 전처리 설정
transform = transforms.Compose([transforms.ToTensor()])

# MNIST 데이터셋 다운로드
train_datasets=datasets.MNIST(root='./data',
                               train=True, transform=transform, download=True)
test_datasets=datasets.MNIST(root='./data',
                              train=False, transform=transform, download=True)

# 데이터 로더 설정
train_loader = DataLoader(train_datasets, batch_size=32, shuffle=True)
test_loader = DataLoader(test_datasets, batch_size=32, shuffle=False)
```

설명

```
# datasets는 MNIST 데이터 다운받을때 그리고
# transforms는 데이터를 전처리 하는 라이브러리, 이미지를 텐서로 변환할때 사용
# DataLoader는 여러개의 이미지를 하나의 batch로 묶을때 사용

# MNIST 데이터셋 다운로드
# train=True 면 이미지 6만장 False면 1만장이 들어옴
# 둘다 ./data에 받아도 pytorch가 학습용/테스트용을 알아서 구분해서 불러옴

# 데이터 로더 설정
# train_datasets에서 로드를 하는데 32개의 샘플을 하나의 배치로 로딩하고
# 데이터셋 (예: MNIST 전체 60,000장 이미지)을 한 번에 모두 처리하지 않고,
# 한 번에 32개씩 나누어서 (=batch) 모델에 입력해서 학습/테스트하겠다는 뜻
# (트레이닝을 더 잘하기 위함)
# shuffle을 통해서 데이터를 섞어서 불러오겠다는 뜻 (학습 효율을 위해)
# 이미지 32장을 한 묶음으로 모아서 처리한다는 뜻
```

그다음 실행 버튼 눌러서 다운로드 하자.

다운로드가 다 되었다면 data라는 폴더가 생길 것이다.

MNIST 데이터 출력해보기 (Matplotlib 사용)

데이터가 잘 다운로드 되었는지 출력해 볼 것이다.

우리는 데이터를 시각화하기 위해 Matplotlib 모듈을 설치할 것이다.

```
# pip install matplotlib 설치 하기

pip install matplotlib
```

```
import matplotlib.pyplot as plt
```

```
# 다운받은 MNIST 샘플 이미지 데이터 출력해보기
sample_data = train_datasets[0]
sample_image, sample_label = sample_data

plt.imshow(sample_image.squeeze(), cmap='gray')

plt.title(sample_label)

plt.show()
```

설명

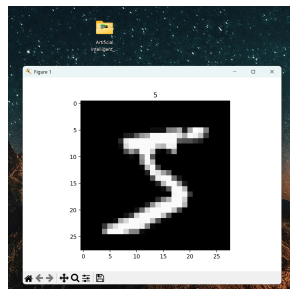
```
# .pyplot은 matplotlib에서 그래프를 간단히 그릴 수 있는 모듈이다.

# 이전에 다운로드된 데이터 중 첫번째 샘플을 꺼낸다
# (image_tensor, label) 튜플 형태 반환

# sample_image의 형태
# sample_image.shape # torch.Size([1, 28, 28])
# 1: 채널수(channel) (MNIST는 흑백을 1채널이라 한다) / 28, 28: 이미지 높이와 너비

# 그런데 matplotlib.pyplot.imshow()는 배열(28x28) [28,28]형태를 받는다.
# imshow()는 이미지를 시각화 하는 함수로 어떤 이미지를 보여줄지 정의하는 함수다.
# .squeeze() : 이미지 1장 보여주는데 필요 없는 값 제거
# cmap='gray' : colormap = 'gray'로 하면 이미지를 흑백 이미지로 시각화

# .title() : 이미지 위에 라벨을 제목으로 올려준다.
# show()로 이미지 출력
```



Pytorch를 사용해서 신경망 모델 구축

인공신경망의 종류에는 MLP, CNN, RNN 그리고 Trasformer 등이 있다.

그 중에서 우리는 MLP (Multi-Layer Perceptron, 다층 퍼셉트론)를 먼저 실습해 볼 것이다.

MLP는 CNN RNN의 근간이 되는 인공신경망으로, 입력층-은닉층-출력층으로 이루어진 가장 기본적인 완전연결형 인공신경망이다.

- 입력층 : 모델의 입력 데이터를 제공하는 층으로 이미지 하나를 넣으면 28*28 개의 픽셀이 모두 입력층으로 들어갈 것이다.
- 은닉층: 입력층과 출력층 사이에 있으며, 입력층으로 받은 값을 복잡한 데이터로 패턴을 학습하는 층이다.
 - 각 은닉층에서는 **가중치(weight)**와 **편향(bias)**를 적용한 뒤, **활성화 함수(Activation function)**를 통해 비선형성을 추가한다.
 - 여기서 ReLU(Rectified Linear Unit)를 많이 사용한다:
 - 입력이 0보다 크면 그대로 출력
 - 입력이 0 이하이면 0 출력
 - 여러 개의 은닉층을 쌓으면 복잡한 패턴을 학습할 수 있다.
- 출력층: 은닉층의 결과를 바탕으로 최종 예측되는 결과를 출력한다.

인공신경망 모델을 만들어 보자.

```
import torch
import torch.nn as nn

class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(28*28, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):

        x = torch.flatten(x, 1)
        x = self.fc1(x)
        x = nn.functional.relu(x)
        x = self.fc2(x)
        return x
```

설명

```
# nn모듈의 생성자 함수 상속
# fc1, fc2라고 하는 두개의 레이어 정의해주기
# fully connected Layer 를 줄여서 fc라고 했다 (공식약어)

# nn.Linear는 신경망의 "층(layer)"을 정의하는 클래스이다.
# 입력 차원과 출력 차원을 지정해서, 입력 데이터를 선형변환(가중치·편향 적용)한다.
# 예: 28*28 픽셀(=784차원 입력)을 받아 128차원 벡터로 변환하는 레이어.
# 즉, 784개의 원소를 128개의 특징 벡터(feature vector)로 압축/가공하는 과정.
# 128은 하이퍼파라미터로, 보통 2의 제곱수(128, 256 등)를 많이 사용한다.

# forward (순전파)
# 신경망의 nn.Linear(완전연결층)는 2차원 입력 [배치크기, 특징차원]만 받는다.
# 따라서 CNN이나 이미지 데이터를 MLP에 넣으려면 꼭 flatten(평탄화 작업)이 필요하다.

# 예시) x.shape = [32, 1, 28, 28]
# - 32 = 배치 크기(이미지 32장)
# - 1 = 채널 수(흑백 이미지)
# - 28x28 = 가로x세로 픽셀 수

# x = torch.flatten(x, 1)을 하면 x.shape = [32, 784]이 된다.
# - 32 = 배치 크기 그대로 유지
# - 784 = 1x28x28 픽셀을 1차원 벡터로 펼친 값

# 이후 fc1 레이어에 넣어 128차원 벡터로 변환하고,
# ReLU 활성화 함수를 거쳐 비선형성을 추가한 뒤,
# fc2 레이어에 넣어 가중치(W), 편향(b)를 학습해 최종 10개 분류 클래스에 대한 출력을 만든다.
```

인공신경망 모델을 학습시켜 보자.

모델학습이란?

오차를 최소화 하는 방향으로 파라미터를 조절하는 과정이다.

학습 과정은 다음과 같다.

```
입력 → 순전파 → 예측값 → 손실 계산 → 역전파 → 최적화(가중치 업데이트)
      ↑                                     ↓
      └── gradient 초기화 (zero_grad) ───┘
                        ↑
                        └── 학습 반복 ───┘
```

1. 순전파

SimpleNN 모델에서 정의한 forward() 메서드를 거쳐 순전파가 이루어 질 것이다.

입력 (28×28 픽셀)
↓
입력층 (784 뉴런에 가중치·편향 적용)
↓
은닉층 1 → ReLU (활성화함수 적용)
↓
은닉층 2 → ReLU (활성화함수 적용)
↓
출력층 (예: 10개 클래스 점수)

2. 손실계산

그리고 손실을 계산할 것이다. loss값을 구하는 손실함수의 종류는 여러가지가 있다.

그 중에서 우리는 cross-entropy 손실함수를 사용하겠다

- 손실 측정을 위한 corss-entropy (크로스엔트로피) 손실함수
 - 딥러닝을 이용한 분류문제에서 가장 널리 쓰이는 손실함수다.
 - 마지막 출력층이 softmax로 0~9 (10개 클래스)에 대해 확률값을 출력할 때 대표적으로 쓰는 손실 함수다. softmax 란 모델이 최종적으로 출력한 값의 확률(probability) 분포로 모델이 어떤 클래스일 확률이 가장 높은지 알려준다.
 - 즉, 크로스엔트로피 손실함수는 모델을 테스트 결과값의 확률분포와 실제 정답의 확률분포 차이를 잘 측정해 준다.
 - 크로스엔트로피 손실값이 낮을수록 실제 정답에 가까운 예측을 하고 있다는 뜻이다.

출력층의결과(예측값)과 정답라벨을 가지고 손실계산
↓
손실함수(Cross-Entropy)를 통해 손실값(Loss)계산한다.

3. 역전파 (Backward propagation)

역전파를 통해서 손실함수값 Loss를 가지고 기울기를 구할 것이다.

손실값을 출력층 → 은닉층 → 입력층 순서로 기울기(gradient)를 전파한다.
미분을 통해 각 층의 가중치와 편향이 얼마나 바뀌어야 하는지 계산이 된다.

4. 가중치 업데이트 (Optimization)

최적화를 할 때에는 SGD (확률적 경사하강법, Stochastic Gradient Descent)을 사용할 것이다. 이는 머신러닝 딥러닝에서 많이 사용하는 최적화 알고리즘 중 하나다

- SGD는 손실함수의 값을 줄이기 위해 신경망 파라미터를 반복적으로 업데이트 해준다.
- 손실을 계산후 손실값이 적어지도록 경사(gradient)를 계산해서 가중치를 업데이트 한다.

기존 가중치에 SGD/Adam 등을 사용해서 gradient 적용
가중치 그리고 편향값을 조금씩 수정한다.

5. gradient 초기화를 진행한다.

- optimizer.zero_grad()를 통해서 gradient를 초기화 한다. (안 하면 gradient가 누적됨)

이렇게 데이터의 한 batch로(묶음) 학습을 하는 것을 1 step 이라 하며

준비된 모든 데이터를 이용해서 여러번의 step을 마치는 것을 1 epoch 이라고 한다.

예를들어 MNIST로 부터 모델훈련에 사용할 이미지를 다운받으면 60000장이 다운로드가 된다.

한번에 60000장을 한번 학습하는것 보다 60000만장을 32장을 하나의 묶음으로 1875번 반복해서 학습을 하면서 매개변수를 조정해 나간다. 이때, 32 장 한 묶음을 학습하는 과정이 1 step이 되며 1875번 스텝으로 60000장 이미지의 학습과정이 다 마치면 1Epoch이 되는 것이다.

우리가 정의한 모델을 학습시켜 보자.

그리고 학습된 모델을 파일로 저장할 것이다.

```
import torch.optim as optim #최적화를 위한 라이브러리

net = SimpleNN()
optimizer = optim.SGD(net.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()

for epoch in range(10):
    for i, data in enumerate(train_loader):
        inputs, labels = data

        # 기울기 초기화
        optimizer.zero_grad()

        # 순전파
        outputs = net(inputs)

        # 손실계산
        loss = criterion(outputs, labels)

        # 역전파
        loss.backward()

        # 옵티마이저
        optimizer.step()
        print(f'Epoch: {epoch}, Loss:{loss.item()}')

torch.save(net, 'model.pt')
```

설명

```
# net의 파라미터를 최적화 해줄 것이다.
# SGD(확률적 경사하강법)을 이용해서 최적화를 할 것이다.
# lr은 학습률을 이야기 하는데 가중치를 바꿀때 1%씩 바꿔보자는 뜻이다.
# 이 값이 너무 크면 빨리 학습하지만 최적의 결과를 갖지 못하는데
# 보통 0.01을 많이 사용한다.

# 손실함수 정의 (크로스엔트로피 사용)

# 총 10에폭 훈련시킬 것이다 (6만장을 10번)
# 훈련 루프 (1step = 32batch/60000만장이미지 = 1875번 학습)
# 결과적으로 1875 step을 10번 한다는 이야기 이다.

# input-신경망에 넣을 이미지데이터로 0~1사이의 픽셀값
# labels-손실계산때 사용할 정답(숫자)

# torch.save통해서 pytorch객체를 파일로 저장하는 함수다.
# net은 내가 만든 신경망 모델 이름이고, model.pt는 파일명이다.
# .pt는 pytorch Tensor 줄임말로 pytorch모델 파일을 저장한다는 약어로 많이 쓰임
```

에폭을 많이 돌릴 수록 loss값은 점점 줄어들 것이다.

그리고 학습된 모델이 파일로 잘 저장되어 있는지도 확인하자.

학습된 모델 평가

학습된 모델을 평가하는 방법

평가지표로는 정확도(Accuracy) / 정밀도(Precision) / 재현률 (Recall) 등이 있다.

- 정확도
 - 올바르게 예측한 샘플수 // 전체 샘플수

- 직관적이고 해석하기 쉽다
- 모집단의 분포가 불균형한 상태에서는 잘못된 정보를 제공할 수 있다.
- 정밀도
 - 양성으로 예측된 것 중 실제로 양성인 것의 비율
 - 진짜양성 (TP) // (진짜양성 (TP)+거짓양성(FP))
 - 예시 : “ 로봇아 네가 지우개라고 골라낸 것 중에서 진짜 지우개는 얼마나 있니?”
- 재현율
 - 실제 양성 중 얼마나 많은 양성을 올바르게 예측 하였는지 확인
 - 진짜양성 (TP) // (진짜양성 (TP) + 거짓음성(FN))
 - 예시 : “ 100명의 학생 중 실제 코로나 환자가 10명인데 코로나 환자가 7명이라고 하면 재현율 70%”
 - 정확도는 97%가 되는 것이다.

우리가 하고있는 실습은 0~9 까지의 분포가 균형 잡혀 있기 때문에 간단하게 정확도를 사용해 학습된 모델을 평가하겠다.

evaluation.py 파일을 새로 만들자.

```
from MLP_model import SimpleNN, test_loader
import torch

# net = torch.load('model.pt', weights_only=False)
net = SimpleNN()
net.load_state_dict(torch.load('model.pt'))

net.eval()

correct = 0
total = 0

with torch.no_grad():
    for data in test_loader:
        images, labels = data

        outputs = net(images)

        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)

        correct += ((predicted == labels).sum().item())

accuracy = 100*correct/total
print(f'accuracy:{accuracy}')
```

설명

```
# 네트워크 가져오기
# weights_only=False를 넣어줘야 함
# 이전에는 전체 객체 (모델 클래스 정의 + 파라미터)를 불러왔으나,
# 최신 버전에서는 "파라미터(가중치)만" 로드하는 것이 기본
# 따라서 전체 객체를 불러오려면 weights_only=False 을 넣어줘야 한다.

# PyTorch 모델을 “평가 모드(evaluation mode)”로 전환
# 특정 레이어들은 학습모드, 평가모드가 서로 다르게 작동하는 레이어가 있다.
# 또한 평가모드에서는 우리가 저장한 학습된 모델이 6만장이 아닌 1만장의 MNIST 이미지로
# 평가를 한다. 그래서 지금은 평가모드라고 알려주는 것이다.
```

```
# with torch.no_grad():
# 이 안에서는 미분 계산이 필요 없으니 autograd 추적을 꺼줘

# 이미지를 신경망에 넣어 얻은 결과를

# torch.max()를 이용해 텐서의 최대값을 구한다.
# _는 파이썬에서 "값은 있지만 쓰지 않겠다"는 관례적 변수명
# predicted : 최댓값이 나온 인덱스 (확률이 가장 높게 나온 숫자를 의미)

# 이미지 샘플 개수의 총합을 구해서
# print(labels.size())의 결과값은 torch.Size([32])
# .size(0)을 사용하면 배치의 데이터 개수 확인가능

# predicted = tensor([1, 0, 4, 9])
# labels     = tensor([1, 2, 4, 8])
# print(predicted == labels)
# true 1 or false 0의 개수를 더한 후
# .item()을 해야 Pytorch의 텐서값을 파이썬 정수로 바꿔줌
```

실제 예제로 테스트 (숫자분류테스트)

실제 숫자 이미지를 넣어서 내 모델의 정확도를 확인해 보자.

이미지 3개를 만들어 보자. 그림판으로 검은 배경에 흰색 숫자를 적어서 만들자.

그리고 mydata라는 폴더를 만들어서 안에 넣어두자.

myimg_test.py 파일을 하나 만들자.

그리고 파이썬에서 이미지 다룰때 사용하는 pillow를 설치하자.

```
pip install pillow
```

```
from PIL import Image
from MLP_model import SimpleNN
import torch
from torchvision import transforms

net = torch.load('model.pt', weights_only=False)
net.eval()

img_path = './mydata/9.png'
img = Image.open(img_path).convert('L') # 흑백모드 변환
img = img.resize((28, 28))

transform = transforms.Compose([transforms.ToTensor()])

img = transform(img).unsqueeze(0)

with torch.no_grad():
    output = net(img)
    print(output)
    _, predicted = torch.max(output.data, 1)

print(f'prediction: {predicted.item()}')
```

설명

```
# img = transform(img).unsqueeze(0)

# 이미지 전처리하고
# .unsqueeze(0)은 텐서의 0번째 (맨 앞) 차원에 크기 1짜리 새로운 차원을 추가
```

```
# 즉, (C, H, W) 형상 → (1, C, H, W)로 바뀐다 (3차원 -> 4차원)
# 여기서 1은 "배치(batch)" 차원을 말한다.

# 신경망에 이미지를 "하나"를 넣을 때도,
# 일반적으로 "배치 단위(묶음)"로 넣어야 하므로
# 크기를 (배치, 채널, 높이, 너비) 형태로 맞춰주는 역할이다.
```

[image 텐서 변천사]

```
* MLP (완전연결 신경망, nn.Linear) 인 경우 [배치 크기, 784] → MNIST (28×28 이미지)
* CNN (이미지 모델)인 경우 [N, C, H, W] (배치, 채널, 높이, 너비)
```

우리가 업로드한 이미지 또는 MNIST에서 다운받은 이미지의 구조는 다음과 같다.

이미지: [H, W, C] (높이, 너비, 채널) 이지만

PyTorch의 기본 이미지 텐서구조는 : [C, H, W] 이다

신경망 모델은 보통 입력을 배치 단위로 받는다.

그래서 이미지 1장도 [C, H, W] → [1, C, H, W]로 바꿔줘야한다.

이때, `unsqueeze(0)` 을 통해서 바꿔준 것이다.

이전에 MNIST 에서 다운받은 이미지를 생각해 보자.

1. 이전에 MNIST에서 다운로드 받은 이미지를 [H,W,C]
2. `transforms.Compose([transforms.ToTensor()])` 통해서 [C,H,W] 로 바꾸고
3. DataLoader에 넣으면 [배치크기, 채널, 높이, 너비] 스타일로 바꿔주고
[N,C,H,W] = [32,1,28,28]
4. 우리가 만든 Simple_NN 모델 안에
`x = torch.flatten(x, 1)` 을 통해서 (N, C, H, W) → (N, C*H*W) MLP 스타일로 바꿈

정리를 하자면

```
* PIL 이미지 → [H, W, C]
* ToTensor() → [C, H, W]
* DataLoader or unsqueeze(0) → [N, C, H, W]
* Flatten → [N, C*H*W]
```

정확도가 좋지 않은 것 같다. 대충 학습시킨 결과라고 본다.

이미지처리에 특화된 CNN 모델을 사용하면 조금 더 좋지 않을까 싶다.